

MEGASCON Hash function

Submission to ICCS NGCC

Wenquan Bi Patrick Derbez Xiaoyang Dong*
Pierre-Alain Fouque Keting Jia Lingyue Qin
André Schrottenloher Liyuan Tang
Boxin Zhao

*Corresponding submitter: Xiaoyang Dong
Tsinghua University, Beijing, P.R.China
xiaoyangdong@tsinghua.edu.cn

Tsinghua University, Beijing, P.R.China
Zhongguancun Laboratory, Beijing, P.R.China
Univ Rennes, Inria, CNRS, IRISA, Rennes, France

September 28, 2026

Contents

1	Introduction	3
2	Specification	3
2.1	Notations	3
2.2	Parameters	3
2.3	Sponge function and Feed-forward Sponge	4
2.4	Permutation	6
2.4.1	Substitution Layer	6
2.4.2	Bit Permutation	7
2.4.3	Linear Diffusion Layer	7
2.4.4	Addition of Constants	8
2.5	Domain separation	8
3	Design Rationale	8
3.1	Design of the Permutation	8
3.1.1	Choice of the S-box Operation	8
3.1.2	Choice of Bit Permutation	9
3.1.3	Choice of the linear layer	9
4	Security Claims	10
5	Security Analysis	10
5.1	Theoretical Security Analysis	10
5.2	Practical Security Analysis	10
5.2.1	Security against linear and differential cryptanalysis . . .	10
5.2.2	Meet-in-the-middle Attack	11
5.3	Quantum Analysis	11
5.4	Randomness Analysis	12
5.5	Resistance to Length Extension Attacks	12
6	Features	13
7	Implementation	14
7.1	SIMD and Parallel implementation	14
7.1.1	S-boxes	14
7.1.2	Row-mix	16
7.1.3	Round Constant	17
7.2	Resource-constrained Implementation	17
7.3	Software Implementation Performance	18
7.4	Hardware	18
A	Randomness Analysis Result	21

1 Introduction

MEGASCON is a permutation-based design inspired by the ASCON family, but adapted to operate on a significantly larger 2048-bit state. The state is organized as an 8×256 bitslice matrix, with the goal of preserving efficient software execution while providing a larger security margin suitable for post-quantum settings.

Its nonlinear layer is based on an extension of the χ transformation to inputs with an even number of bits, as introduced in CHILow. This allows Megascon to retain a simple and low-latency Boolean structure while operating naturally on 8-bit vertical slices of the state.

The linear layer is inspired by Ascon’s diffusion strategy, but is adapted to work efficiently on 256-bit rows. It relies on rotations and XOR operations over 64-bit words within each row, making it well suited to bitsliced software implementations on modern processors. Altogether, Megascon combines Ascon-like design principles with a wider state and a diffusion layer tailored for high-performance software. MEGASCON works in two modes of operations, *i.e.*, the Sponge function [5] and the feed-forward Sponge [9, 15].

2 Specification

2.1 Notations

a/b	number of rounds of the permutation, $a = 20$ and $b = 24$
r	size of rate or outer part in bit
c	size of capacity or inner part in bit
h	sizes of the digest in bits
$\text{MSB}_s(x)$	the most significant s bits of x
$\text{LSB}_s(x)$	the least significant s bits of x

2.2 Parameters

MEGASCON is a family of hash functions, which has 4 variants with digest bit-size $h=384, 512, 768$, and 1024 bits, denoted as (MEGASCON-384, MEGASCON-512, MEGASCON-768, and MEGASCON-1024). Besides, two extendable output functions (XOFs) to produce hash outputs of arbitrary length are defined as MEGASCON-384-XOF and MEGASCON-512-XOF.

- For MEGASCON-384 and MEGASCON-512, the hash functions are built on the Sponge function [5], which is defined as $\text{SPONGE}_{h,r,a}(M, \ell)$ parameterized by the digest length h , the rate r , the number of rounds a of the permutation p^a (p is a single round of the permutation), the input message M and output bitsize ℓ (if $h \neq 0, \ell = h$). $\text{SPONGE}_{h,r,a}(M, \ell)$ maps an input message M of arbitrary length to a fixed-length hash output T :

$$\text{SPONGE}_{h,r,a}(M, \ell) = T, \text{ where } |T| = h \text{ bits.} \quad (1)$$

- For MEGASCON-768 and MEGASCON-1024, the hash functions are built on the feed-forward Sponge function [9], which is defined as $\text{F-SPONGE}_{h,r,b}(M)$ parameterized by the digest length h , the rate r , the number of rounds b of the permutation p^b (p is a single round of the permutation), the input message M . $\text{F-SPONGE}_{h,r,b}(M)$ maps an input message M of bitsize smaller than 2^{64} to a fixed-length hash output T :

$$\text{F-SPONGE}_{h,r,b}(M) = T, \text{ where } |T| = h \text{ bits.} \quad (2)$$

- For MEGASCON-384-XOF and MEGASCON-512-XOF, the sponge function $\text{SPONGE}_{h,r,a}(M, \ell)$ is applied with $h = 0$. The output is of arbitrary bit-size ℓ .

$$\text{SPONGE}_{0,r,a}(M, \ell) = T, \text{ where } |T| = \ell \text{ bits.} \quad (3)$$

Table 1 summarizes the recommended instances for hash functions and and XOF with specified parameters.

Table 1: Parameters for recommended hashing functions and XOFs

Name	Algorithm	Bit size of			state	Rounds
		digest (h)	capacity(c)	rate (r)		
MEGASCON-384	$\text{SPONGE}_{384,1280,20}(M, \ell)$	384	768	1280	2048	15
MEGASCON-512	$\text{SPONGE}_{512,1024,20}(M, \ell)$	512	1024	1024	2048	15
MEGASCON-768	$\text{F-SPONGE}_{768,1216,24}(M)$	768	832	1216	2048	16
MEGASCON-1024	$\text{F-SPONGE}_{1024,960,24}(M)$	1024	1088	960	2048	16
MEGASCON-384-XOF	$\text{SPONGE}_{0,1280,20}(M, \ell)$	arbitrary	768	1280	2048	15
MEGASCON-512-XOF	$\text{SPONGE}_{0,1024,20}(M, \ell)$	arbitrary	1024	1024	2048	15

2.3 Sponge function and Feed-forward Sponge

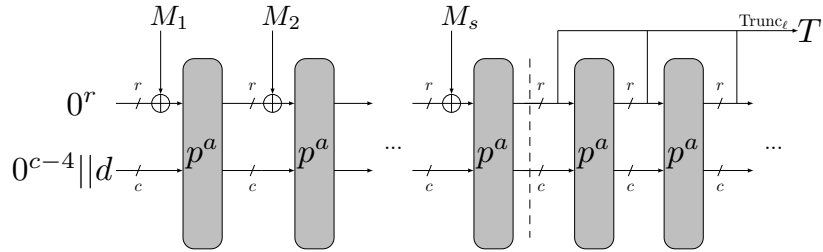


Figure 1: Sponge function

Sponge function $\text{SPONGE}_{h,r,a}(M, \ell)$. The Sponge structure is given in Figure 1 and $\text{SPONGE}_{h,r,a}(M, \ell)$ is specified in Algorithm 1. In the initialization phase, 4-bit domain encoding number d is fed in the last four bits of initial state for different version to make domain separation. The details are shown in Sect. 2.5.

Algorithm 1: The Sponge function $\text{SPONGE}_{h,r,a}(M, \ell)$

Input: Message $M \in \{0, 1\}^*$, output bitsize $\ell \leq h$ or arbitrary if $h = 0$
Output: Digest $T \in \{0, 1\}^\ell$
 /* Initialization, 4-bit domain encoding d is specified in Sect. 2.5 Table 2 for different ℓ */
 1 $\mathcal{I} \leftarrow 0^{2044} \| d$
 /* Absorbing */
 2 $M_1, \dots, M_s \leftarrow M \| 1 \| 0^*$, where the bitsize of M_i ($1 \leq i \leq s$) is r bits
 3 **for** $i = 1$ **to** s **do**
 4 $\mathcal{I} \leftarrow p^a(\mathcal{I} \oplus (M_i \| 0^{2048-r}))$
 /* Squeezing */
 5 $\lambda \leftarrow \lfloor \frac{\ell}{r} \rfloor$
 6 $Z \leftarrow \text{MSB}_r(\mathcal{I})$
 7 **for** $i = 1$ **to** λ **do**
 8 $\mathcal{I} \leftarrow p^a(\mathcal{I})$
 9 $Z \leftarrow Z \| \text{MSB}_r(\mathcal{I})$
 10 **return** $T = \text{MSB}_\ell(Z)$

Feed-forward Sponge Function $\text{F-SPONGE}_{h,r,b}(M)$. The feed-forward sponge construction [15, 9] is given in Figure 2, and $\text{F-SPONGE}_{1024,960,24}(M)$ is specified in Algorithm 2. In the initialization phase, 4-bit domain encoding number d is fed in the last four bits of initial state for different version to make domain separation. The details are shown in Sect. 2.5.

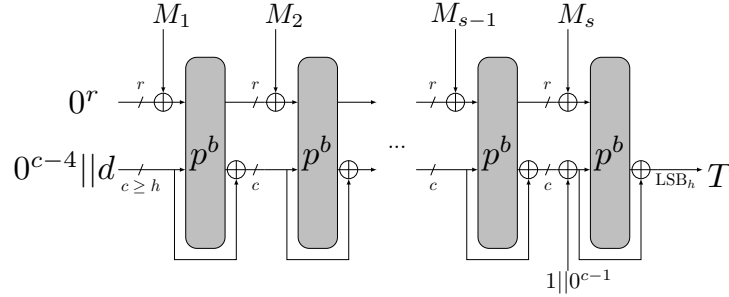


Figure 2: Feed-forward Sponge function

Algorithm 2: The feed-forward Sponge function $\text{F-SPONGE}_{h,r,b}(M)$

Input: Message $M \in \{0,1\}^*$
Output: Digest $T \in \{0,1\}^h$
 /* Initialization, 4-bit domain encoding d is specified in Sect. 2.5 Table 2 for different h */
 1 $\mathcal{I} \leftarrow 0^{2044} \| d$
 /* Absorbing */
 2 $M_1, \dots, M_s \leftarrow M \| 1 \| 0^*$, where the bitsize of M_i ($1 \leq i \leq s$) is r bits
 3 **for** $i = 1$ **to** s **do**
 4 **if** $i \neq s$ **then**
 5 $\mathcal{I} \leftarrow \mathcal{I} \oplus (M_i \| 0^{2048-r})$
 6 **else**
 7 $\mathcal{I} \leftarrow \mathcal{I} \oplus (M_i \| 1 \| 0^{2048-r-1})$
 8 $\mathcal{I} \leftarrow (0^r \| \text{LSB}_{2048-r}(\mathcal{I})) \oplus p^b(\mathcal{I})$
 /* Squeezing */
 9 $T \leftarrow \text{LSB}_h(\mathcal{I})$
 10 **return** T

2.4 Permutation

The main component of Megascon is a 2048-bit permutation p composed of four steps:

$$p = p_C \circ p_L \circ p_\pi \circ p_S.$$

The state is represented as an 8×256 bitslice matrix

$$S = (x_0, x_1, \dots, x_7),$$

where each row x_i is a 256-bit word. Each row is further decomposed into four 64-bit words:

$$x_i = x_{i,0} \parallel x_{i,1} \parallel x_{i,2} \parallel x_{i,3}.$$

The external bit string is mapped to the state in a row-wise order. More precisely, for an external bit index j , the corresponding state bit is $x_i[\ell]$, where

$$j = 256i + \ell, \quad 0 \leq i \leq 7, \quad 0 \leq \ell \leq 255.$$

Equivalently, the state is traversed row by row, from x_0 to x_7 . This ordering is used when the rate part of the state is filled during absorbing and read during squeezing.

2.4.1 Substitution Layer

The substitution layer p_S applies 256 parallel instances of an 8-bit S-box to the columns of the state. For each column position j , the bits

$$(x_0[j], x_1[j], \dots, x_7[j])$$

are updated using the 8-bit χ -like transformation introduced in CHiLOW for even-dimensional inputs. With input bits $(u_0, \dots, u_7)$ and output bits $(v_0, \dots, v_7)$, the S-box is defined by

$$\begin{aligned} v_0 &= u_0 \oplus (\neg u_1 \wedge u_2), \\ v_1 &= u_4 \oplus (\neg u_2 \wedge u_0), \\ v_2 &= u_3 \oplus (\neg u_0 \wedge u_1), \\ v_3 &= \neg u_1 \oplus (\neg u_4 \wedge \neg u_5), \\ v_4 &= u_2 \oplus (\neg u_5 \wedge u_6), \\ v_5 &= u_5 \oplus (\neg u_6 \wedge u_7), \\ v_6 &= u_6 \oplus (\neg u_7 \wedge u_3), \\ v_7 &= u_7 \oplus (\neg u_3 \wedge u_4). \end{aligned}$$

2.4.2 Bit Permutation

After the substitution layer, a fixed permutation p_π is applied to the eight rows of the bitsliced state. The permutation is

$$\pi = (1, 7, 5, 2, 4, 6, 0, 3),$$

meaning that row i is moved to row $\pi(i)$:

$$x_{\pi(i)} \leftarrow x_i, \quad 0 \leq i < 8.$$

2.4.3 Linear Diffusion Layer

The linear diffusion layer p_L acts independently on each 256-bit row. For row i , let

$$x_i = x_{i,0} \parallel x_{i,1} \parallel x_{i,2} \parallel x_{i,3},$$

where each $x_{i,j}$ is a 64-bit word. The output words are defined by

$$\begin{aligned} y_{i,j} &= x_{i,j} \oplus (x_{i,j+1} \lll a_i) \oplus (x_{i,j+2} \lll b_i) \\ &\quad \oplus (x_{i,j+3} \lll c_i) \oplus (x_{i,j} \lll d_i), \end{aligned}$$

where the word indices are taken modulo 4, and $\lll$ denotes rotation to the left inside a 64-bit word.

The rotation constants are

i	a_i	b_i	c_i	d_i
0	1	3	14	36
1	2	5	23	60
2	4	15	17	24
3	7	9	12	40
4	26	46	13	37
5	49	6	50	25
6	55	19	56	57
7	58	47	61	28

2.4.4 Addition of Constants

The constant addition step p_C xors a round-dependent 64-bit constant c_r into the first word of the first row:

$$x_{0,0} \leftarrow x_{0,0} \oplus c_r.$$

The constants c_r are generated independently for each round and are used to break symmetries between rounds.

2.5 Domain separation

We use the last four bits in IV to make the domain separation. That is, in the initialization phase, the initial state is assigned by concatenating 2044 bits of zeros with the 4-bit domain encoding, which is located in the least significant 4 bits in capacity. The domain encoding for different versions is listed in Table 2. Different versions have different domain encode numbers fed in the last four bits of IV . For example, the input IV value is 2044 bit zeros, concatenating 4 bits 0001 for MEGASCON-384 in the initialization phase, while for MEGASCON-384-XOF, the four bit number is 1001.

Table 2: Domain encoding for different hash and XOF versions

Versions	MEGASCON -384	MEGASCON -512	MEGASCON -768	MEGASCON -1024	MEGASCON -384-XOF	MEGASCON -512-XOF
Domain values	0001	0011	0100	0101	1001	1011

3 Design Rationale

3.1 Design of the Permutation

3.1.1 Choice of the S-box Operation

The $\chi\chi$ S-box used in MEGASCON inherits its structure from Daemen’s χ transformation, while extending it to even dimensions as introduced in **ChiLow** [3]. This construction is particularly well suited for low-latency implementations. Indeed, each output bit is expressed as a simple quadratic Boolean function involving only one AND operation and a small number of XOR and NOT operations. As a result, the overall circuit depth is very small, since the nonlinear part can be evaluated in essentially a single layer of AND gates followed by linear operations. This type of structure is known to achieve optimal or near-optimal latency complexity for Boolean functions implemented over standard gate sets such as $\{\text{AND}, \text{XOR}, \text{NOT}\}$, where latency is driven by the longest dependency path in the circuit.

Moreover, the $\chi\chi$ construction preserves the locality and regularity of the original χ function, meaning that each output bit depends only on a small

number of neighboring input bits. This leads to highly parallelizable implementations and avoids long dependency chains, which are the main source of latency in hardware and bitsliced software. In addition, $\chi\chi$ naturally defines a permutation in even dimension, allowing to work on states of size a power of 2.

Altogether, the combination of low algebraic degree, minimal multiplicative complexity, and short dependency chains makes $\chi\chi$ an excellent candidate for low-latency S-box design, especially in wide bitsliced permutations such as MEGASCON.

3.1.2 Choice of Bit Permutation

We chose to apply a bit permutation at the output of the S-box, to remove some fixed points in the DDT. More precisely, $\chi\chi$ has exactly 8 differential transitions for which both the input and the output have only one active bit. The permutation was chosen to ensure there is no cycle of length shorter than 8 when iterating such transitions. Among all the bit permutations achieving this property, we selected one minimizing the cycles while looking as well at differential transitions $2 \rightarrow 1$ and $1 \rightarrow 2$.

3.1.3 Choice of the linear layer

The linear diffusion layer of MEGASCON is designed to balance strong diffusion with efficient software implementation on wide words. A natural starting point is the linear layer used in ASCON, which relies on two rotations and two XOR operations per word. However, this structure proves insufficient in our setting: when extended to 256-bit rows, it diffuses too slowly across the state, requiring a large number of rounds to propagate differences. A straightforward extension to three rotations and three XORs would improve diffusion, but such a construction is not invertible due to the even-parity structure of the resulting linear mapping. For this reason, we adopt a design with four rotations and four XOR operations, ensuring invertibility while significantly improving the speed of diffusion.

From an implementation perspective, the design is tailored to modern software platforms. While there is no direct instruction to rotate a full 256-bit word, most architectures provide efficient instructions to rotate the four 64-bit lanes of a 256-bit register independently. In addition, instructions are available to permute or shuffle these 64-bit words at low cost. Our linear layer leverages exactly these primitives: rotations are applied within each 64-bit word, while cross-word diffusion is achieved through word permutations, leading to an efficient and regular bitsliced implementation.

Finally, the rotation parameters are carefully selected to maximize diffusion. We require all rotation offsets to be distinct in order to avoid structural symmetries and ensure good mixing. The parameters are then chosen through an extensive search, optimizing several criteria: fast propagation of differences (measured through saturation experiments), strong diffusion properties, and good resistance to clustering effects. In particular, we favor parameter sets that achieve near-optimal diffusion and exhibit the best collision-free behavior after a

small number of rounds (typically four), which empirically correlates with good security margins.

4 Security Claims

We provide our security claims for the different variants of MEGASCON in Table 3. Basically, for all versions of MEGASCON, we claim that it is hard to find a collision with substantially less than $2^{h/2}$ hash evaluations, and hard to find a (second-)preimage with substantially less than 2^h hash evaluations, where h denotes the digest size.

Table 3: Security claims for recommended parameters of MEGASCON. For second preimage security spre., security is considered for challenge messages with up to 2^{64} blocks (after padding).

Requirement	Security in bits					
	MEGASCON -384	MEGASCON -512	MEGASCON -768	MEGASCON -1024	MEGASCON -384-XOF	MEGASCON -512-XOF
Collision	192	256	384	512	$\min(384, \ell/2)$	$\min(512, \ell/2)$
Preimage	384	512	768	1024	$\min(384, \ell)$	$\min(512, \ell)$
2nd preimage	384	512	768	1024	$\min(384, \ell)$	$\min(512, \ell)$

5 Security Analysis

5.1 Theoretical Security Analysis

The mode of operation of MEGASCON-384, MEGASCON-512, MEGASCON-384-XOF, and MEGASCON-512-XOF is based on the sponge methodology proposed by Bertoni et al. [5] and profits from the extensive literature on Sponges, particularly the results on its indistinguishability up to about $2^{c/2}$ calls to the permutation or its inverse, where c is the capacity in bits [5].

The mode of operation of MEGASCON-768, MEGASCON-1024 is based on the feed-forward sponge methodology proposed by Guo et al. [9] and Sun et al. [15] with formal security proofs.

5.2 Practical Security Analysis

5.2.1 Security against linear and differential cryptanalysis

As expected, the large state size of MEGASCON makes its analysis particularly challenging and opens several directions for future work. Despite significant efforts, we were not able to derive tight bounds on the probability or correlation of the best differential or linear trails even for three rounds.

To nevertheless gain insight into the diffusion properties of the permutation, we adopted a greedy search strategy. Starting from low-weight input differences, we iteratively construct trails by optimizing, at each round, the number of

active S-boxes. While this approach does not guarantee optimality, it provides meaningful lower bounds on the number of active S-boxes over multiple rounds.

Using this method, the best trails we identified over 10 rounds involve 997 active S-boxes. Extending the search to 11 rounds, we were unable to find any trail with fewer than 1135 active S-boxes. These results strongly suggest that any differential characteristic over a sufficient number of rounds must involve a very large number of active S-boxes. Consequently, it appears highly unlikely that differential attacks could lead to collisions more efficiently than generic attacks.

5.2.2 Meet-in-the-middle Attack

The MEGASCON permutation achieves full diffusion with 4 rounds, which guarantees that any bit non-linearly affects all bits after 4 rounds in the forward direction. According to Qin *et al.*'s MitM attack model on Sponge function [12] and the partial matching principle [13], the possible number of rounds for partial matching is $(4-1)=3$ rounds. The number of possible rounds for the initial structure is $(4-1)=3$ rounds. Then, the possible number of rounds used for the MitM attack is estimated as $(4-1)+(4-1)+1=7$ rounds for MEGASCON-384, MEGASCON-512, MEGASCON-384-XOF, and MEGASCON-512-XOF that based on the sponge, leaving a security margin of $15-7=8$ rounds.

For MEGASCON-768, MEGASCON-1024 based on the feed-forward sponge, we evaluate the security against MitM attack following a framework of splice-and-cut technique [13], which is also adopted by the self-evaluation of SKINNY [2] and MIDORI [1]. The possible number of rounds for the partial matching [13] is estimated as $(4-1)+(4-1)+1=7$ rounds. The number of possible rounds for the initial structure is $(4-1)=3$ rounds. The splice-and-cut may extend the number of attack rounds up to the smaller number of full diffusion rounds minus one. Then, possible number of rounds used for MitM is $7+3+3=13$ rounds, leaving a security margin of $16-13=3$ rounds.

5.3 Quantum Analysis

Given a hash function with h -bit digest, the quantum preimage or 2nd preimage attacks should be not efficient than the Grover's algorithm [8], whose time complexities is $2^{h/2}$. At CRYPTO 2022, Schrottenloher and Stevens Marc[14] proposed the quantum preimage attack by introducing the quantum MitM attack. The quantum attack leverages the valid MitM trails. However, as analyzed in Sect. 5.2.2, no MitM trails cover the full rounds of MEGASCON permutation. Therefore, there is no quantum MitM preimage attack on full MEGASCON.

The complexity of generic quantum collision-finding algorithms is highly dependent on the resources available to the adversary. Existing literature typically considers three scenarios, distinguished by the type and scale of memory and computing resources, each leading to different time-space tradeoffs.

- **Scenario 1.** Assuming a polynomial-size quantum computer with access to exponentially large quantum random access memory (qRAM), the

BHT algorithm [6] achieves both time and qRAM complexity $2^{h/3}$. While this setting yields optimal time complexity for generic quantum collision finding, the requirement for large-scale qRAM poses significant practical challenges.

- **Scenario 2.** In the absence of exponentially large quantum or classical memory, the quantum analogue of the parallel rho method [16, 4, 10] employs S quantum processors to achieve a time-space tradeoff of $2^{h/2}/S$, making it suitable for memory-constrained settings with parallel quantum resources.
- **Scenario 3.** Allowing large classical memory while avoiding exponentially large qRAM, the CNS algorithm [7] finds collisions in time $2^{2h/5}$ using $2^{h/5}$ classical RAM, providing a balanced tradeoff between efficiency and resource availability.

According to our analysis in Sect. 5.2.1, there is no full-round differential trials which beat any of the three generic quantum collision attacks.

5.4 Randomness Analysis

We evaluate the randomness of the outputs using 15 statistical tests, including *MonoBitFrequencyTest*, *FrequencyWithinBlockTest*, *PokerTest*, *OverlappingTemplateMatchingTest*, *RunsTest*, *RunsDistributionTest*, *LongestRunOfOnesInABlockTest*, *BinaryDerivativeTest*, *AutocorrelationTest*, *MatrixRankTest*, *CumulativeTest*, *ApproximateEntropyTest*, *LinearComplexityTest*, *MaurerUniversalTest*, and *DiscreteFourierTransformTest*. For each test, the significance level is set to $\alpha = 0.01$. A sequence is considered to pass a test if the corresponding p -value is greater than or equal to α . The detailed randomness analysis results are given in Section A.

Our experimental results show that, for all input types (random, high Hamming weight, and low Hamming weight), the outputs consistently satisfy this criterion across all tests. Consequently, the outputs are shown to pass all statistical randomness tests.

5.5 Resistance to Length Extension Attacks

Since we employ two hash constructions, namely the sponge and the feed-forward sponge structures, for different combinations of r and h , we provide separate explanations for the resistance of each construction against length extension attacks.

1. **SPONGE:** For the sponge construction, resistance to length extension attacks relies on the fact that the inner part of the state remains hidden during the squeezing phase. In particular, only the outer part is released as output, while the c -bit inner part is never exposed to the adversary. As a result, the adversary cannot recover the full internal state, and thus cannot continue the computation to perform a length extension attack.

2. **F-SPONGE:** For the F-sponge construction, resistance to length extension attacks is ensured by a domain separation mechanism. As stated by [9], directly outputting the inner part may enable length-extension attacks. To prevent this, when the last padded message block M_s is XORed into the outer part of the state, a constant $\mathbb{1} = 0^{c-1}\|1$ is simultaneously XORed into the inner part.

Therefore, MEGASCON is resistant to length extension attacks.

6 Features

The main feature of MEGASCON is the balance of the security, performance and implement cost.

High security margin. MEGASCON adopts the sponge construction and Feed-forward Sponge construction which have a lot of security research and rigorous security proofs. The permutation MEGASCON used also has rigorous security margin, *i.e.*, the limited active S-box numbers. Both of them ensure the security of MEGASCON.

Performance. MEGASCON adopts the Feed-forward Sponge construction for larger digest versions and has a relatively large rate ratio compared to classical sponge construction, which makes it to absorb more messages and achieve a high through output. It also can be greatly optimized by the vector registers and SIMD instructions in the modern CPU.

Flexibility All operations of the permutation used by MEGASCON can be implemented in a bit-slice fashion and can be computed in parallel. This makes it flexible to make a balance in the performance and costs.

Innovativeness MEGASCON adopts the sponge construction and the feed-forward Sponge construction for different length of digest. To support 1024 bits digest, there will be a very large state for popular hash mode sponge construction. To make the state smaller relatively, we choose the feed-forward Sponge construction for 768 and 1024 bit digests, keeping good performance and security at the same time. In the aspect of internal permutation, we design bit-slice and parallel friendly components in Ascon-like style to achieve good security and performance especially in modern CPUs which supporting SIMD instructions. The longer the vector register length, the better the algorithm performance.

7 Implementation

7.1 SIMD and Parallel implementation

MEGASCON is designed in a bit-slice fashion, and the main components can be implemented in parallel. For maximum parallelization, SIMD-based implementations yield better performance. The following functions are used in the SIMD implementation of MEGASCON.

7.1.1 S-boxes

For software implementations, the substitution layer p_S is evaluated in a bit-sliced manner. Instead of applying the 8-bit S-box to each column independently, the same Boolean operations are applied to vector words. In this representation, the eight vectors ($\text{row}[0], \dots, \text{row}[7]$) contain the eight input bit planes of many S-box instances. Hence, one vectorized evaluation of the Boolean formulas defining the S-box computes many S-boxes in parallel.

In the SSE implementation, the Boolean operations are realized by vectorized XOR, AND-NOT, and NOT operations; see Listing 1.

Listing 1: SSE implementation of the bit-sliced S-box.

```
1 static inline void chichi_sse_regs(Vec256_SSE row[8]) {
2     Vec256_SSE x0 = row[0], x1 = row[1], x2 = row[2], x3 = row[3];
3     Vec256_SSE x4 = row[4], x5 = row[5], x6 = row[6], x7 = row[7];
4
5     row[BIT_PERM[0]] = xor256(x0, andnot256(x1, x2));
6     row[BIT_PERM[1]] = xor256(x4, andnot256(x2, x0));
7     row[BIT_PERM[2]] = xor256(x3, andnot256(x0, x1));
8     row[BIT_PERM[3]] = xor256(not256(x1), andnot256(x4, not256(x5)));
9     row[BIT_PERM[4]] = xor256(x2, andnot256(x5, x6));
10    row[BIT_PERM[5]] = xor256(x5, andnot256(x6, x7));
11    row[BIT_PERM[6]] = xor256(x6, andnot256(x7, x3));
12    row[BIT_PERM[7]] = xor256(x7, andnot256(x3, x4));
13 }
```

The AVX2 implementation follows the same bit-sliced structure; see Listing 2. Since AVX2 does not provide a dedicated bitwise NOT instruction, the negation is implemented by XORing with an all-one vector.

Listing 2: AVX2 implementation of the bit-sliced S-box.

```
1 static inline void chichi_avx2_regs(__m256i row[8]) {
2     __m256i x0 = row[0], x1 = row[1], x2 = row[2], x3 = row[3];
3     __m256i x4 = row[4], x5 = row[5], x6 = row[6], x7 = row[7];
4
5     const __m256i all1 =
6         _mm256_cmpeq_epi32(_mm256_setzero_si256(), _mm256_setzero_si256())
7         );
```

```

8   row[BIT_PERM[0]] = _mm256_xor_si256(x0, _mm256_andnot_si256(x1, x2));
9   row[BIT_PERM[1]] = _mm256_xor_si256(x4, _mm256_andnot_si256(x2, x0));
10  row[BIT_PERM[2]] = _mm256_xor_si256(x3, _mm256_andnot_si256(x0, x1));
11
12  __m256i nx1 = _mm256_xor_si256(x1, all1);
13  __m256i nx5 = _mm256_xor_si256(x5, all1);
14  row[BIT_PERM[3]] = _mm256_xor_si256(nx1, _mm256_andnot_si256(x4, nx5)
15  );
16
17  row[BIT_PERM[4]] = _mm256_xor_si256(x2, _mm256_andnot_si256(x5, x6));
18  row[BIT_PERM[5]] = _mm256_xor_si256(x5, _mm256_andnot_si256(x6, x7));
19  row[BIT_PERM[6]] = _mm256_xor_si256(x6, _mm256_andnot_si256(x7, x3));
20  row[BIT_PERM[7]] = _mm256_xor_si256(x7, _mm256_andnot_si256(x3, x4));
21 }

```

On processors supporting AVX-512 ternary-logic instructions, in particular AVX-512VL for 256-bit vectors, each three-input Boolean function can be evaluated by a single ternary-logic instruction. The corresponding AVX-512 implementation of the bit-sliced S-box is given in Listing 3. In this implementation, the immediate values 0xa6 and 0x56 encode the Boolean functions $c \oplus (\neg a \wedge b)$ and $\neg c \oplus (\neg a \wedge \neg b)$, respectively.

Listing 3: AVX-512 implementation of the bit-sliced S-box using ternary logic.

```

1  static inline __m256i chi3_avx512(__m256i a, __m256i b, __m256i c) {
2      return _mm256_ternarylogic_epi64(a, b, c, 0xa6);
3  }
4
5  static inline __m256i chi3_special_avx512(__m256i a, __m256i b, __m256i c
6      ) {
7      return _mm256_ternarylogic_epi64(a, b, c, 0x56);
8  }
9
10 static inline void chichi_avx512_regs(__m256i row[8]) {
11     __m256i x0 = row[0], x1 = row[1], x2 = row[2], x3 = row[3];
12     __m256i x4 = row[4], x5 = row[5], x6 = row[6], x7 = row[7];
13
14     row[BIT_PERM[0]] = chi3_avx512(x1, x2, x0);
15     row[BIT_PERM[1]] = chi3_avx512(x2, x0, x4);
16     row[BIT_PERM[2]] = chi3_avx512(x0, x1, x3);
17     row[BIT_PERM[3]] = chi3_special_avx512(x4, x5, x1);
18     row[BIT_PERM[4]] = chi3_avx512(x5, x6, x2);
19     row[BIT_PERM[5]] = chi3_avx512(x6, x7, x5);
20     row[BIT_PERM[6]] = chi3_avx512(x7, x3, x6);
21     row[BIT_PERM[7]] = chi3_avx512(x3, x4, x7);
22 }

```

7.1.2 Row-mix

In the SIMD implementation, the bit permutation layer p_π and the linear diffusion layer p_L are implemented together. Recall that p_π only permutes the eight bit-sliced rows, while p_L acts independently on each row. Therefore, instead of first computing all rows of p_L and then applying p_π , we compute the linear diffusion of each row and directly store the result into its permuted destination.

More precisely, for each row i , the implementation computes

$$y_i = L_i(x_i),$$

where L_i denotes the row-wise linear diffusion with rotation constants (a_i, b_i, c_i, d_i) . The result is then written to row $\pi(i)$, namely

$$\text{row}[\pi(i)] \leftarrow y_i.$$

In this way, the row permutation and the row-wise linear diffusion are combined into a single row-mix step.

For a 256-bit SIMD vector

$$x_i = x_{i,0} \parallel x_{i,1} \parallel x_{i,2} \parallel x_{i,3},$$

the row-wise linear diffusion is implemented by forming three word-rotated copies of the vector and applying 64-bit rotations inside each word. The common SIMD implementation pattern is given in Listing 4.

Listing 4: Common SIMD implementation pattern of the row-mix operation.

```

1 Vec256 row_mix_one(Vec256 x, unsigned a, unsigned b,
2                     unsigned c, unsigned d) {
3     Vec256 x1 = word_rot1(x);
4     Vec256 x2 = word_rot2(x);
5     Vec256 x3 = word_rot3(x);
6     Vec256 xd = rotl64(x, d);
7
8     return x ^ xd
9         ^ rotl64(x1, a)
10        ^ rotl64(x2, b)
11        ^ rotl64(x3, c);
12 }
```

Here, `word_rot1`, `word_rot2`, and `word_rot3` cyclically rotate the four 64-bit words of the SIMD vector by one, two, and three positions, respectively, while `rotl64` applies a left rotation independently to each 64-bit word.

The SSE, AVX2, and AVX-512 implementations all follow this pattern. They differ only in the instruction-level realization of `word_rot1`, `word_rot2`, `word_rot3`, and `rotl64`. In particular, the AVX2 implementation uses 64-bit lane permutations with immediate values such as `0x39`, `0x4e`, and `0x93` to re-align the three word rotations. The SSE and AVX-512 implementations use the corresponding shuffle or permutation routines available in those instruction sets.

Table 4: 16 round constants generated by Listing 5.

Round r	Round constant c_r
0	0xe220a8397b1dcdaf
1	0xe4d971771b652c20
2	0x975835de1c9756ce
3	0x910a2dec89025cc1
4	0xc164bb7b2b9b3e3a
5	0x16b1cba95fc60262
6	0xf3203e9039f4a821
7	0xf75f04cbb5a1a1dd
8	0x9e5651b0ef953636
9	0x63cbe1e459320dd7
10	0x088712be8a582fca
11	0xaeaf52febe706064
12	0x6e73e372e2338aca
13	0x1d0b14e4db018fed
14	0xbd64a5d9adefe000
15	0x63033b0ca389c35a

7.1.3 Round Constant

In the software implementation, the round constants are generated by applying a 64-bit mixing function to the round index. The generation procedure is given in Listing 5.

Listing 5: Generation of round constants.

```

1 uint64_t round_constant(unsigned round) {
2     uint64_t x = 0x9e3779b97f4a7c15 ^ (uint64_t)round;
3     x ^= x >> 30;
4     x *= 0xbf58476d1ce4e5b9;
5     x ^= x >> 27;
6     x *= 0x94d049bb133111eb;
7     x ^= x >> 31;
8     return x;
9 }
```

The 16 round constants generated by this procedure are listed in Table 4.

7.2 Resource-constrained Implementation

Since MEGASCON is designed in a bit-sliced fashion, the implementation is similar to the SIMD implementation in resource-constrained environments. The substitution layer p_S and bit permutation p_π are processed with several words and have no interference with each other. The only part that requires extra processing is the linear diffusion layer p_L operation, while there is already a lot of research such as [11] and it is not a very complex task.

Table 5: Software performance for 1000-byte messages on the Intel Xeon Gold 6548N platform. Throughput is given in MB/s.

Algorithm	SSE	AVX2	AVX-512
MEGASCON-384	297.16	524.01	809.39
MEGASCON-512	260.91	471.07	694.19
MEGASCON-768	276.20	490.55	764.16
MEGASCON-1024	217.91	383.49	611.15
SHA3-384	274.12	291.66	402.72
SHA3-512	201.00	207.07	290.51

Table 6: Software performance for 100 MB messages on the Intel Xeon Gold 6548N platform. Throughput is given in MB/s.

Algorithm	SSE	AVX2	AVX-512
MEGASCON-384	347.46	621.03	1013.65
MEGASCON-512	277.06	499.29	806.55
MEGASCON-768	307.71	554.20	894.55
MEGASCON-1024	242.84	437.02	702.58
SHA3-384	316.19	309.67	443.25
SHA3-512	219.00	214.62	307.50

7.3 Software Implementation Performance

We evaluate the software performance of MEGASCON on a server equipped with Intel(R) Xeon(R) Gold 6548N processors, with a maximum CPU frequency of 4.10 GHz. The operating system is Ubuntu 22.04 LTS. The implementation is single-threaded and compiled by GCC 11.4.0 with the `-O3` optimization option. Three instruction-set variants are tested: SSE, AVX2, and AVX-512. For comparison, we also evaluate SHA3-384 and SHA3-512 using the implementation from XKCP under the same platform and testing methodology.

For each parameter set, two types of messages are tested. The first test uses 1000-byte messages repeated 1,000,000 times (Table 5), and the second test uses 100 MB messages repeated 10 times (Table 6). Therefore, the total amount of processed data is 1000 MB in both tests. The average throughput is computed as the total number of processed bytes divided by the elapsed time.

7.4 Hardware

We evaluate the ASIC performance of MEGASCON using Synopsys Design Compiler with the NanGate 15 nm and NanGate 45 nm standard-cell libraries. The SPONGE structure is used for MEGASCON-384 and MEGASCON-512, and the F-SPONGE structure is used for MEGASCON-768 and MEGASCON-1024. The throughput is calculated based on the corresponding rate and the number of cycles per message block. The results are shown in Tables 7 and 8.

Table 7: ASIC synthesis results of MEGASCON on the NanGate 15 nm library.

Algorithm	Structure	Critical Path	Area	Cycles/block	Throughput
MEGASCON-384	SPONGE	68.45 ps	11764.33 μm^2	15	1246.7 Gbps
MEGASCON-512	SPONGE	68.45 ps	11764.33 μm^2	15	997.4 Gbps
MEGASCON-768	F-SPONGE	72.39 ps	14889.37 μm^2	16	1049.8 Gbps
MEGASCON-1024	F-SPONGE	72.39 ps	14889.37 μm^2	16	828.8 Gbps

Table 8: ASIC synthesis results of MEGASCON on the NanGate 45 nm library.

Algorithm	Structure	Critical Path	Area	Cycles/block	Throughput
MEGASCON-384	SPONGE	0.647 ns	47902.34 μm^2	15	131.96 Gbps
MEGASCON-512	SPONGE	0.647 ns	47902.34 μm^2	15	105.57 Gbps
MEGASCON-768	F-SPONGE	0.720 ns	59552.35 μm^2	16	105.53 Gbps
MEGASCON-1024	F-SPONGE	0.720 ns	59552.35 μm^2	16	83.31 Gbps

References

- [1] Subhadeep Banik, Andrey Bogdanov, Takanori Isobe, Kyoji Shibutani, Harunaga Hiwatari, Toru Akishita, and Francesco Regazzoni. Midori: A block cipher for low energy. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 411–436. Springer, 2015.
- [2] Christof Beierle, Jérémy Jean, Stefan Kölbl, Gregor Leander, Amir Moradi, Thomas Peyrin, Yu Sasaki, Pascal Sasdrich, and Siang Meng Sim. The skinny family of block ciphers and its low-latency variant mantis. In *Annual international cryptology conference*, pages 123–153. Springer, 2016.
- [3] Yanis Belkheyar, Patrick Derbez, Shibam Ghosh, Gregor Leander, Silvia Mella, Léo Perrin, Shahram Rasoolzadeh, Lukas Stennes, Siwei Sun, Gilles Van Assche, and Damian Vizár. Chilow and chichi: New constructions for code encryption. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part I*, Lecture Notes in Computer Science, pages 212–243. Springer, 2025.
- [4] Daniel J Bernstein. Cost analysis of hash collisions: Will quantum computers make sharcs obsolete. *SHARCS*, 9:105, 2009.
- [5] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sponge functions. In *ECRYPT hash workshop*, volume 2007, 2007.
- [6] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. In *LATIN 1998*, pages 163–169. Springer, 1998.

- [7] André Chailloux, María Naya-Plasencia, and André Schrottenloher. An efficient quantum collision search algorithm and implications on symmetric cryptography. In *ASIACRYPT 2017*, volume 10625, pages 211–240. Springer, 2017.
- [8] Lov K Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 212–219, 1996.
- [9] Chun Guo, Kai Hu, Yanhong Fan, Yong Fu, and Meiqin Wang. Adding feeding forward back to the sponge construction. Cryptology ePrint Archive, Paper 2025/1006, 2025.
- [10] Akinori Hosoyamada and Yu Sasaki. Finding hash collisions with quantum computers by using differential trails with smaller probability than birthday bound. In *EUROCRYPT 2020*, pages 249–279. Springer, 2020.
- [11] Junhao Huang, Alexandre Adomnicăi, Jipeng Zhang, Wangchen Dai, Yao Liu, Ray CC Cheung, Çetin Kaya Koç, and Donglong Chen. Revisiting keccak and dilithium implementations on armv7-m. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(2):1–24, 2024.
- [12] Lingyue Qin, Jialiang Hua, Xiaoyang Dong, Hailun Yan, and Xiaoyun Wang. Meet-in-the-middle preimage attacks on sponge-based hashing. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 158–188. Springer, 2023.
- [13] Yu Sasaki and Kazumaro Aoki. Finding preimages in full md5 faster than exhaustive search. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 134–152. Springer, 2009.
- [14] André Schrottenloher and Marc Stevens. Simplified mitm modeling for permutations: New (quantum) attacks. In *Annual International Cryptology Conference*, pages 717–747. Springer, 2022.
- [15] Siwei Sun, Shun Li, Zhiyu Zhang, Charlotte Lefevre, Bart Mennink, Zhen Qin, and Dengguo Feng. Permutation-based hashing with stronger (second) preimage resistance - application to hash-based signature schemes. Cryptology ePrint Archive, Paper 2025/963, 2025.
- [16] Paul C Van Oorschot and Michael J Wiener. Parallel collision search with cryptanalytic applications. *Journal of cryptology*, 12(1):1–28, 1999.

A Randomness Analysis Result

Table 9: GMT 0005-2021 Randomness Test Results of MOZI-384 with fully random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	990	10
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	989	11
<i>PokerTest</i> ($m = 4$)	991	9
<i>PokerTest</i> ($m = 8$)	988	12
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	988	12
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	991	9
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	984	16
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	989	11
<i>RunsTest</i>	987	13
<i>RunsDistributionTest</i>	990	10
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	985	15
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	995	5
<i>BinaryDerivativeTest</i> ($k = 3$)	986	14
<i>BinaryDerivativeTest</i> ($k = 7$)	982	18
<i>AutocorrelationTest</i> ($d = 1$)	987	13
<i>AutocorrelationTest</i> ($d = 2$)	994	6
<i>AutocorrelationTest</i> ($d = 8$)	989	11
<i>AutocorrelationTest</i> ($d = 16$)	989	11
<i>MatrixRankTest</i>	992	8
<i>CumulativeTest</i> (Forward)	986	14
<i>CumulativeTest</i> (Backward)	989	11
<i>ApproximateEntropyTest</i> ($m = 2$)	988	12
<i>ApproximateEntropyTest</i> ($m = 5$)	983	17
<i>LinearComplexityTest</i> ($m = 500$)	990	10
<i>LinearComplexityTest</i> ($m = 1000$)	990	10
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	985	15
<i>DiscreteFourierTransformTest</i>	989	11

Table 10: GMT 0005-2021 Randomness Test Results of MOZI-384 with high Hamming weight random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	989	11
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	991	9
<i>PokerTest</i> ($m = 4$)	991	9
<i>PokerTest</i> ($m = 8$)	990	10
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	986	14
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	988	12
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	992	8
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	992	8
<i>RunsTest</i>	989	11
<i>RunsDistributionTest</i>	994	6
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	993	7
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	991	9
<i>BinaryDerivativeTest</i> ($k = 3$)	987	13
<i>BinaryDerivativeTest</i> ($k = 7$)	984	16
<i>AutocorrelationTest</i> ($d = 1$)	989	11
<i>AutocorrelationTest</i> ($d = 2$)	989	11
<i>AutocorrelationTest</i> ($d = 8$)	986	14
<i>AutocorrelationTest</i> ($d = 16$)	991	9
<i>MatrixRankTest</i>	995	5
<i>CumulativeTest</i> (Forward)	992	8
<i>CumulativeTest</i> (Backward)	989	11
<i>ApproximateEntropyTest</i> ($m = 2$)	986	14
<i>ApproximateEntropyTest</i> ($m = 5$)	995	5
<i>LinearComplexityTest</i> ($m = 500$)	989	11
<i>LinearComplexityTest</i> ($m = 1000$)	991	9
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	988	12
<i>DiscreteFourierTransformTest</i>	985	15

Table 11: GMT 0005-2021 Randomness Test Results of MOZI-384 with low Hamming weight random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	992	8
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	987	13
<i>PokerTest</i> ($m = 4$)	992	8
<i>PokerTest</i> ($m = 8$)	986	14
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	993	7
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	992	8
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	986	14
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	987	13
<i>RunsTest</i>	991	9
<i>RunsDistributionTest</i>	988	12
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	995	5
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	992	8
<i>BinaryDerivativeTest</i> ($k = 3$)	989	11
<i>BinaryDerivativeTest</i> ($k = 7$)	991	9
<i>AutocorrelationTest</i> ($d = 1$)	991	9
<i>AutocorrelationTest</i> ($d = 2$)	994	6
<i>AutocorrelationTest</i> ($d = 8$)	993	7
<i>AutocorrelationTest</i> ($d = 16$)	995	5
<i>MatrixRankTest</i>	987	13
<i>CumulativeTest</i> (Forward)	994	6
<i>CumulativeTest</i> (Backward)	990	10
<i>ApproximateEntropyTest</i> ($m = 2$)	993	7
<i>ApproximateEntropyTest</i> ($m = 5$)	992	8
<i>LinearComplexityTest</i> ($m = 500$)	996	4
<i>LinearComplexityTest</i> ($m = 1000$)	993	7
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	982	18
<i>DiscreteFourierTransformTest</i>	991	9

Table 12: GMT 0005-2021 Randomness Test Results of MOZI-512 with fully random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	991	9
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	990	10
<i>PokerTest</i> ($m = 4$)	994	6
<i>PokerTest</i> ($m = 8$)	987	13
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	995	5
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	993	7
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	988	12
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	991	9
<i>RunsTest</i>	989	11
<i>RunsDistributionTest</i>	991	9
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	986	14
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	991	9
<i>BinaryDerivativeTest</i> ($k = 3$)	988	12
<i>BinaryDerivativeTest</i> ($k = 7$)	989	11
<i>AutocorrelationTest</i> ($d = 1$)	989	11
<i>AutocorrelationTest</i> ($d = 2$)	996	4
<i>AutocorrelationTest</i> ($d = 8$)	993	7
<i>AutocorrelationTest</i> ($d = 16$)	989	11
<i>MatrixRankTest</i>	993	7
<i>CumulativeTest</i> (Forward)	985	15
<i>CumulativeTest</i> (Backward)	991	9
<i>ApproximateEntropyTest</i> ($m = 2$)	995	5
<i>ApproximateEntropyTest</i> ($m = 5$)	987	13
<i>LinearComplexityTest</i> ($m = 500$)	991	9
<i>LinearComplexityTest</i> ($m = 1000$)	991	9
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	990	10
<i>DiscreteFourierTransformTest</i>	987	13

Table 13: GMT 0005-2021 Randomness Test Results of MOZI-512 with high Hamming weight random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	991	9
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	992	8
<i>PokerTest</i> ($m = 4$)	995	5
<i>PokerTest</i> ($m = 8$)	986	14
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	992	8
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	990	10
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	992	8
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	990	10
<i>RunsTest</i>	982	18
<i>RunsDistributionTest</i>	992	8
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	995	5
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	989	11
<i>BinaryDerivativeTest</i> ($k = 3$)	988	12
<i>BinaryDerivativeTest</i> ($k = 7$)	990	10
<i>AutocorrelationTest</i> ($d = 1$)	982	18
<i>AutocorrelationTest</i> ($d = 2$)	994	6
<i>AutocorrelationTest</i> ($d = 8$)	988	12
<i>AutocorrelationTest</i> ($d = 16$)	990	10
<i>MatrixRankTest</i>	987	13
<i>CumulativeTest</i> (Forward)	990	10
<i>CumulativeTest</i> (Backward)	989	11
<i>ApproximateEntropyTest</i> ($m = 2$)	992	8
<i>ApproximateEntropyTest</i> ($m = 5$)	989	11
<i>LinearComplexityTest</i> ($m = 500$)	989	11
<i>LinearComplexityTest</i> ($m = 1000$)	990	10
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	989	11
<i>DiscreteFourierTransformTest</i>	990	10

Table 14: GMT 0005-2021 Randomness Test Results of MOZI-512 with low Hamming weight random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	988	12
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	986	14
<i>PokerTest</i> ($m = 4$)	992	8
<i>PokerTest</i> ($m = 8$)	987	13
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	994	6
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	992	8
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	994	6
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	991	9
<i>RunsTest</i>	988	12
<i>RunsDistributionTest</i>	988	12
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	993	7
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	988	12
<i>BinaryDerivativeTest</i> ($k = 3$)	988	12
<i>BinaryDerivativeTest</i> ($k = 7$)	992	8
<i>AutocorrelationTest</i> ($d = 1$)	988	12
<i>AutocorrelationTest</i> ($d = 2$)	990	10
<i>AutocorrelationTest</i> ($d = 8$)	989	11
<i>AutocorrelationTest</i> ($d = 16$)	993	7
<i>MatrixRankTest</i>	990	10
<i>CumulativeTest</i> (Forward)	988	12
<i>CumulativeTest</i> (Backward)	986	14
<i>ApproximateEntropyTest</i> ($m = 2$)	994	6
<i>ApproximateEntropyTest</i> ($m = 5$)	994	6
<i>LinearComplexityTest</i> ($m = 500$)	995	5
<i>LinearComplexityTest</i> ($m = 1000$)	991	9
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	988	12
<i>DiscreteFourierTransformTest</i>	988	12

Table 15: GMT 0005-2021 Randomness Test Results of MOZI-768 with fully random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	990	10
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	989	11
<i>PokerTest</i> ($m = 4$)	992	8
<i>PokerTest</i> ($m = 8$)	995	5
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	992	8
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	990	10
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	995	5
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	989	11
<i>RunsTest</i>	990	10
<i>RunsDistributionTest</i>	991	9
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	987	13
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	989	11
<i>BinaryDerivativeTest</i> ($k = 3$)	991	9
<i>BinaryDerivativeTest</i> ($k = 7$)	994	6
<i>AutocorrelationTest</i> ($d = 1$)	990	10
<i>AutocorrelationTest</i> ($d = 2$)	993	7
<i>AutocorrelationTest</i> ($d = 8$)	990	10
<i>AutocorrelationTest</i> ($d = 16$)	992	8
<i>MatrixRankTest</i>	986	14
<i>CumulativeTest</i> (Forward)	992	8
<i>CumulativeTest</i> (Backward)	993	7
<i>ApproximateEntropyTest</i> ($m = 2$)	992	8
<i>ApproximateEntropyTest</i> ($m = 5$)	992	8
<i>LinearComplexityTest</i> ($m = 500$)	988	12
<i>LinearComplexityTest</i> ($m = 1000$)	985	15
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	992	8
<i>DiscreteFourierTransformTest</i>	987	13

Table 16: GMT 0005-2021 Randomness Test Results of MOZI-768 with high Hamming weight random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	988	12
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	988	12
<i>PokerTest</i> ($m = 4$)	982	18
<i>PokerTest</i> ($m = 8$)	988	12
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	993	7
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	995	5
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	991	9
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	989	11
<i>RunsTest</i>	985	15
<i>RunsDistributionTest</i>	988	12
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	986	14
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	992	8
<i>BinaryDerivativeTest</i> ($k = 3$)	994	6
<i>BinaryDerivativeTest</i> ($k = 7$)	990	10
<i>AutocorrelationTest</i> ($d = 1$)	985	15
<i>AutocorrelationTest</i> ($d = 2$)	989	11
<i>AutocorrelationTest</i> ($d = 8$)	989	11
<i>AutocorrelationTest</i> ($d = 16$)	996	4
<i>MatrixRankTest</i>	989	11
<i>CumulativeTest</i> (Forward)	987	13
<i>CumulativeTest</i> (Backward)	988	12
<i>ApproximateEntropyTest</i> ($m = 2$)	992	8
<i>ApproximateEntropyTest</i> ($m = 5$)	992	8
<i>LinearComplexityTest</i> ($m = 500$)	991	9
<i>LinearComplexityTest</i> ($m = 1000$)	995	5
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	985	15
<i>DiscreteFourierTransformTest</i>	982	18

Table 17: GMT 0005-2021 Randomness Test Results of MOZI-768 with low Hamming weight random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	994	6
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	988	12
<i>PokerTest</i> ($m = 4$)	993	7
<i>PokerTest</i> ($m = 8$)	990	10
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	995	5
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	996	4
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	995	5
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	997	3
<i>RunsTest</i>	989	11
<i>RunsDistributionTest</i>	987	13
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	989	11
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	994	6
<i>BinaryDerivativeTest</i> ($k = 3$)	991	9
<i>BinaryDerivativeTest</i> ($k = 7$)	985	15
<i>AutocorrelationTest</i> ($d = 1$)	989	11
<i>AutocorrelationTest</i> ($d = 2$)	997	3
<i>AutocorrelationTest</i> ($d = 8$)	986	14
<i>AutocorrelationTest</i> ($d = 16$)	994	6
<i>MatrixRankTest</i>	990	10
<i>CumulativeTest</i> (Forward)	992	8
<i>CumulativeTest</i> (Backward)	991	9
<i>ApproximateEntropyTest</i> ($m = 2$)	995	5
<i>ApproximateEntropyTest</i> ($m = 5$)	995	5
<i>LinearComplexityTest</i> ($m = 500$)	991	9
<i>LinearComplexityTest</i> ($m = 1000$)	988	12
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	991	9
<i>DiscreteFourierTransformTest</i>	989	11

Table 18: GMT 0005-2021 Randomness Test Results of MOZI-1024 with high Hamming weight random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	993	7
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	990	10
<i>PokerTest</i> ($m = 4$)	991	9
<i>PokerTest</i> ($m = 8$)	988	12
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	990	10
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	990	10
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	990	10
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	992	8
<i>RunsTest</i>	992	8
<i>RunsDistributionTest</i>	993	7
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	988	12
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	981	19
<i>BinaryDerivativeTest</i> ($k = 3$)	988	12
<i>BinaryDerivativeTest</i> ($k = 7$)	990	10
<i>AutocorrelationTest</i> ($d = 1$)	992	8
<i>AutocorrelationTest</i> ($d = 2$)	989	11
<i>AutocorrelationTest</i> ($d = 8$)	988	12
<i>AutocorrelationTest</i> ($d = 16$)	989	11
<i>MatrixRankTest</i>	991	9
<i>CumulativeTest</i> (Forward)	993	7
<i>CumulativeTest</i> (Backward)	995	5
<i>ApproximateEntropyTest</i> ($m = 2$)	990	10
<i>ApproximateEntropyTest</i> ($m = 5$)	997	3
<i>LinearComplexityTest</i> ($m = 500$)	986	14
<i>LinearComplexityTest</i> ($m = 1000$)	991	9
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	990	10
<i>DiscreteFourierTransformTest</i>	988	12

Table 19: GMT 0005-2021 Randomness Test Results of MOZI-1024 with low Hamming weight random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	989	11
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	991	9
<i>PokerTest</i> ($m = 4$)	989	11
<i>PokerTest</i> ($m = 8$)	994	6
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	991	9
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	988	12
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	989	11
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	990	10
<i>RunsTest</i>	997	3
<i>RunsDistributionTest</i>	987	13
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	994	6
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	991	9
<i>BinaryDerivativeTest</i> ($k = 3$)	984	16
<i>BinaryDerivativeTest</i> ($k = 7$)	991	9
<i>AutocorrelationTest</i> ($d = 1$)	997	3
<i>AutocorrelationTest</i> ($d = 2$)	991	9
<i>AutocorrelationTest</i> ($d = 8$)	989	11
<i>AutocorrelationTest</i> ($d = 16$)	992	8
<i>MatrixRankTest</i>	993	7
<i>CumulativeTest</i> (Forward)	988	12
<i>CumulativeTest</i> (Backward)	988	12
<i>ApproximateEntropyTest</i> ($m = 2$)	991	9
<i>ApproximateEntropyTest</i> ($m = 5$)	992	8
<i>LinearComplexityTest</i> ($m = 500$)	990	10
<i>LinearComplexityTest</i> ($m = 1000$)	983	17
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	987	13
<i>DiscreteFourierTransformTest</i>	990	10